

Network Intrusion Detection System

NSL-KDD Dataset · Python · Scikit-learn

Complete Line-by-Line Code Explanation

This document explains every line of the NIDS project notebook in plain English, covering imports, preprocessing logic, model training, threshold tuning, visualisations, and the live prediction demo.

Table of Contents

Section 1	Imports & Setup
Section 2	Loading the Dataset
Section 3	Binary Labeling
Section 4	Preprocessing
Section 5	Training the Models
Section 5b	Threshold Tuning
Section 6	Evaluation & Visualisations
Section 7	Live Prediction Demo
Section 7b	Mixed-Confidence Demo
Appendix	Why Test Accuracy is ~77% — Expected Behaviour

SECTION 1

Imports & Setup

These lines bring in every external library the project needs. Think of them like loading tools into your workbench before starting a job.

```
import numpy as np
```

NumPy — the core numerical library. Used for array operations, random sampling, and computing statistics like mean and standard deviation.

```
import pandas as pd
```

Pandas — for loading CSV files into DataFrames (spreadsheet-like tables) and manipulating data (filtering rows, adding columns, etc.).

```
import matplotlib.pyplot as plt
```

Matplotlib — Python's standard plotting library. Used to draw all the charts and figures (confusion matrices, ROC curves, bar charts).

```
import seaborn as sns
```

Seaborn — built on top of Matplotlib. Makes prettier statistical plots with less code, especially the heatmaps used for confusion matrices.

```
import warnings; warnings.filterwarnings('ignore')
```

Suppresses harmless warning messages (e.g. convergence warnings from Logistic Regression) so the output stays clean and readable.

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

LabelEncoder: converts text categories (like 'tcp', 'udp') into numbers. StandardScaler: rescales features to have mean=0 and std=1 — required for Logistic Regression.

```
from sklearn.model_selection import cross_val_score
```

A function that performs k-fold cross-validation — splitting the training data into k parts and training/testing k times to get a reliable accuracy estimate.

```
from sklearn.linear_model import LogisticRegression
```

A linear classifier that outputs a probability between 0 and 1. Despite the name, it is used for classification, not regression.

```
from sklearn.tree import DecisionTreeClassifier
```

A tree-based model that splits data by asking a series of yes/no questions on features. Intuitive and fast, but can overfit without depth limits.

```
from sklearn.ensemble import RandomForestClassifier
```

An ensemble of many decision trees trained on random subsets of data and features. More robust than a single tree and gives feature importances.

```
from sklearn.metrics import classification_report, confusion_matrix, ...
```

Evaluation tools: `classification_report` (precision/recall/F1), `confusion_matrix` (actual vs predicted breakdown), `accuracy_score`, `roc_auc_score`, `roc_curve`.

SECTION 2

Loading the Dataset

The NSL-KDD dataset is a classic benchmark for network intrusion detection. It contains labelled network connection records, each described by 41 features.

```
COLUMNS = [  
    'duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes',  
    'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins',  
    ... # 42 columns total  
    'label', 'difficulty'  
]
```

COLUMNS list: The NSL-KDD CSV files have no header row, so we manually provide the 42 column names. The last two are 'label' (attack type e.g. 'normal', 'neptune') and 'difficulty' (not used for modelling).

```
TRAIN_PATH = r'C:\Users\...\KDDTrain.csv'  
TEST_PATH = r'C:\Users\...\KDDTest.csv'  
train_df = pd.read_csv(TRAIN_PATH)  
test_df = pd.read_csv(TEST_PATH)
```

pd.read_csv(): Reads a CSV file and returns a DataFrame. The 'r' prefix on the path string means 'raw string' — backslashes are treated literally instead of as escape characters (important on Windows paths).

```
print(f'Train: {train_df.shape[0]:,} rows | Test: {test_df.shape[0]:,} rows')
```

.shape[0]: Returns the number of rows. The ',' format specifier adds comma separators to large numbers (e.g. 125,973).

Binary Labeling

The original dataset has many attack types (neptune, smurf, portsweep...). We simplify it to a binary problem: Normal (0) or Attack (1).

```
train_df['binary_label'] = (train_df['label'] != 'normal').astype(int)
test_df['binary_label'] = (test_df['label'] != 'normal').astype(int)
```

train_df['label'] != 'normal': Compares every row's label to the string 'normal'. Returns a column of True/False values.

.astype(int): Converts True → 1 (Attack) and False → 0 (Normal). This is called boolean-to-integer casting.

Why binary? Binary classification is simpler and more practical for real-time intrusion detection. We want to know: 'is this traffic suspicious?' not 'which specific attack type is this?'

```
vc = train_df['binary_label'].map({0: 'Normal', 1: 'Attack'}).value_counts()
for k, v in vc.items():
    print(f' {k}: {v:,} ({v/len(train_df)*100:.1f}%)')
```

.map({...}): Replaces numeric values with human-readable strings for display. **.value_counts():** Counts how many rows belong to each class. This helps identify class imbalance (e.g. far more attacks than normal traffic).

Preprocessing

Machine learning models need numbers, not text. This section converts all categorical (text) columns into numeric form and scales the features.

Step A — Label Encoding for small categoricals

```
small_cat_cols = ['protocol_type', 'flag']
le = LabelEncoder()
for col in small_cat_cols:
    combined = pd.concat([train_df[col], test_df[col]])
    le.fit(combined)
    train_df[col] = le.transform(train_df[col])
    test_df[col] = le.transform(test_df[col])
```

```
small_cat_cols = [...]
```

These two columns have very few unique values (`protocol_type` has 3: tcp, udp, icmp; `flag` has 11). Label encoding is acceptable here for tree-based models.

```
le = LabelEncoder()
```

Creates a `LabelEncoder` object. It learns a mapping like: icmp=0, tcp=1, udp=2.

```
combined = pd.concat([train_df[col], test_df[col]])
```

Concatenates both train and test values BEFORE fitting the encoder. This ensures the encoder sees ALL possible category values, not just training ones — preventing errors on unseen test categories.

```
le.fit(combined)
```

Learns the mapping from text to integers across both datasets.

```
le.transform(train_df[col])
```

Applies the learned mapping, converting e.g. 'tcp' → 1 everywhere.

Step B — One-Hot Encoding for 'service'

```
combined_service = pd.concat([train_df['service'], test_df['service']])
service_dummies_all = pd.get_dummies(combined_service, prefix='service')
n_train = len(train_df)
train_service_dummies = service_dummies_all.iloc[:n_train].reset_index(drop=True)
test_service_dummies = service_dummies_all.iloc[n_train:].reset_index(drop=True)
train_df = pd.concat([train_df.drop(columns=['service']).reset_index(drop=True),
train_service_dummies], axis=1)
```

Why one-hot and not label encoding? 'service' has 70+ values like http, ftp, smtp. Label encoding would assign http=1, ftp=2, smtp=3 — implying http < ftp < smtp, which is meaningless. One-hot creates a separate 0/1 column for each service type, with no false ordering.

pd.get_dummies(): Creates one binary column per unique category. If a row has service='http', only the 'service_http' column is 1; all others are 0.

Combined then split: We combine train+test, one-hot encode together, then split back using `iloc[:n_train]` and `iloc[n_train:]`. This guarantees both datasets have identical columns even if some service types only appear in one set.

axis=1: Joins DataFrames column-wise (side by side). `axis=0` would join row-wise (stacking).

Step C — Feature/Target Split & Scaling

```
FEATURE_COLS = [c for c in train_df.columns
if c not in ('label', 'difficulty', 'binary_label')]
X_train = train_df[FEATURE_COLS].values
y_train = train_df['binary_label'].values
X_test = test_df[FEATURE_COLS].values
y_test = test_df['binary_label'].values
```

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

```
FEATURE_COLS = [...]
```

List comprehension that picks every column EXCEPT the label columns. These become the input features (X) for the model.

```
.values
```

Converts a pandas DataFrame/Series to a plain NumPy array. Scikit-learn models work with NumPy arrays.

```
scaler.fit_transform(X_train)
```

`fit()`: learns the mean and std of each feature from training data. `transform()`: subtracts the mean and divides by std. Result: each feature has mean≈0, std≈1.

```
scaler.transform(X_test)
```

Applies the SAME mean/std learned from training data to the test data. IMPORTANT: we never call `fit()` on test data — that would be data leakage.

SECTION 5

Training the Models

Three classifiers are trained in a loop. Each produces predictions, a probability score, and a cross-validation accuracy.

```
models = {  
    'Logistic Regression': (LogisticRegression(max_iter=1000, random_state=42), True),  
    'Decision Tree': (DecisionTreeClassifier(max_depth=15, random_state=42), False),  
    'Random Forest': (RandomForestClassifier(n_estimators=100, n_jobs=-1, random_state=42),  
False),  
}
```

`max_iter=1000`

Logistic Regression uses an iterative solver. 1000 iterations (vs default 100) gives it more time to converge, avoiding ConvergenceWarnings.

`max_depth=15`

Limits how deep the Decision Tree can grow. Without this, the tree memorises the training data (overfitting). Depth 15 is a reasonable cap.

`n_estimators=100`

Number of trees in the Random Forest. More trees = better accuracy + more compute. 100 is a good default.

`n_jobs=-1`

Use ALL available CPU cores to train trees in parallel. Dramatically speeds up Random Forest training.

`random_state=42`

Seeds the random number generator so results are reproducible every run. The number 42 is arbitrary — any integer works.

`True / False (use_scaled)`

Logistic Regression needs scaled features (it's sensitive to feature magnitude). Decision Tree and Random Forest are scale-invariant, so we use raw features.

```
for name, (model, use_scaled) in models.items():  
    Xtr = X_train_sc if use_scaled else X_train  
    Xte = X_test_sc if use_scaled else X_test  
  
    model.fit(Xtr, y_train) # train  
    y_pred = model.predict(Xte) # predict class labels  
    y_prob = model.predict_proba(Xte)[: , 1] # predict probabilities  
  
    acc = accuracy_score(y_test, y_pred)  
    auc = roc_auc_score(y_test, y_prob)  
    cv = cross_val_score(model, Xtr, y_train, cv=5, scoring='accuracy')
```

```
model.fit(Xtr, y_train)
```

Trains the model: it learns the relationship between input features (Xtr) and the output labels (y_train). This is the core learning step.

```
model.predict(Xte)
```

Applies the learned model to test data and returns a predicted label (0 or 1) for every test row.

```
model.predict_proba(Xte)[:, 1]
```

Returns a probability for each class. [:, 1] selects the second column — the probability of being class 1 (Attack). Used for ROC curve and threshold tuning.

```
accuracy_score(y_test, y_pred)
```

Fraction of test rows where the prediction matches the true label. Simple but can be misleading on imbalanced datasets.

```
roc_auc_score(y_test, y_prob)
```

Area Under the ROC Curve. A value of 1.0 is perfect; 0.5 is random guessing. More reliable than accuracy because it considers the full probability range.

```
cross_val_score(..., cv=5)
```

Splits training data into 5 folds, trains on 4 and tests on 1, repeating 5 times. Returns 5 accuracy scores. Mean $\pm$ std shows how stable the model is.

Threshold Tuning

By default, a model predicts 'Attack' only when it is more than 50% confident. In security, missing a real attack is far worse than a false alarm — so we lower the decision threshold.

Why lower the threshold?

Default threshold = 0.5 means: predict Attack only if probability $\geq$ 50%. Lowering to 0.35 means: predict Attack if probability $\geq$ 35%. This catches more real attacks (higher recall) at the cost of a few extra false alarms (lower precision). For a security system, this is the right trade-off.

```
THRESHOLD = 0.35
```

```
best_so_far = max(results, key=lambda k: results[k]['auc'])
y_prob_best = results[best_so_far]['y_prob']
y_pred_tuned = (y_prob_best >= THRESHOLD).astype(int)
```

```
THRESHOLD = 0.35
```

The new decision boundary. Any test sample with attack-probability $\geq$ 0.35 is now flagged as an attack (vs $\geq$ 0.50 before).

```
max(results, key=lambda k: results[k]['auc'])
```

Finds the model with the highest AUC score. `lambda k: ...` is an anonymous function that extracts the AUC value for each model name `k`.

```
y_prob_best >= THRESHOLD
```

Compares each probability to 0.35. Returns True where the model thinks there is $\geq$ 35% chance of an attack.

```
.astype(int)
```

Converts True $\rightarrow$ 1 (predict Attack) and False $\rightarrow$ 0 (predict Normal), giving us the new tuned predictions.

```
print(f'Default (0.5) Attack recall :
{results[best_so_far]["y_pred"][y_test==1].mean():.3f}')
print(f'Tuned (0.35) Attack recall : {y_pred_tuned[y_test==1].mean():.3f}')
```

[y_test==1]: Filters to only the rows that are ACTUALLY attacks in the test set. **.mean():** On a 0/1 array, mean = fraction correctly predicted = recall. Comparing both lines shows how much recall improved by lowering the threshold.

Evaluation & Visualisations

Six plots are arranged in a 2×3 grid: confusion matrices for each model on the top row, and summary charts on the bottom row.

```
fig, axes = plt.subplots(2, 3, figsize=(18, 11))
fig.suptitle('Network Intrusion Detector – NSL-KDD Results',
            fontsize=16, fontweight='bold')
```

plt.subplots(2, 3): Creates a figure with 2 rows and 3 columns of subplots. Returns fig (the whole figure) and axes (a 2D array of subplot panels). `figsize=(18, 11)` sets the width and height in inches.

Row 1 — Confusion Matrices

```
for ax, (name, res), col in zip(axes[0], results.items(), colors):
    cm = confusion_matrix(y_test, res['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['Normal', 'Attack'],
                yticklabels=['Normal', 'Attack'], ax=ax)
    ax.set_title(f'{name}\nAcc: {res["accuracy"]:.3f}', fontsize=11)
```

```
zip(axes[0], results.items(), colors)
```

Iterates three lists in parallel: the 3 subplot axes, the 3 model result dicts, and 3 colours. Each iteration gives one axis, one model, and one colour.

```
confusion_matrix(y_test, res['y_pred'])
```

Returns a 2x2 matrix: [[TN, FP], [FN, TP]]. TN=correctly normal, FP=normal predicted as attack, FN=attack missed, TP=correctly caught.

```
sns.heatmap(cm, annot=True, fmt='d')
```

Draws the matrix as a colour-coded grid. `annot=True` prints the numbers inside each cell. `fmt='d'` formats them as integers (not scientific notation).

Row 2 — Summary Charts

Accuracy vs AUC bar chart: Draws grouped bars for each model. Two bars per model — one for accuracy, one for AUC. The hatch pattern ('/' on AUC bars) helps distinguish them even in black-and-white print.

ROC Curves: Plots the True Positive Rate vs False Positive Rate at every possible threshold. A curve hugging the top-left corner means the model is very good. The diagonal 'k--' line is the baseline (random guessing, AUC=0.5).

Feature Importance (Random Forest): `rf.feature_importances_` gives a score per feature.

`np.argsort(imp)[-15:]` picks the top 15 most important features. `ax.barh()` draws horizontal bars so long feature names (from one-hot encoding) fit without clipping.

Live Prediction Demo

Picks 5 random test rows, predicts each one using the tuned threshold, and shows the actual label, predicted label, confidence, and whether it was correct.

```
np.random.seed(42)
rf_model = results['Random Forest']['model']
idxs = np.random.choice(len(X_test), 5, replace=False)

for i in idxs:
    sample = X_test[i].reshape(1, -1)
    actual = 'Normal' if y_test[i] == 0 else 'Attack'
    prob_attack = rf_model.predict_proba(sample)[0][1]
    pred = 'Attack' if prob_attack >= THRESHOLD else 'Normal'
    conf = prob_attack if pred == 'Attack' else 1 - prob_attack
    status = '■' if actual == pred else '■'
```

```
np.random.seed(42)
```

Fixes the random seed so the same 5 samples are picked every run. Makes the demo reproducible.

```
np.random.choice(len(X_test), 5, replace=False)
```

Picks 5 random index numbers from the test set. replace=False means the same row can't be picked twice.

```
X_test[i].reshape(1, -1)
```

Extracts one row from the test array. reshape(1, -1) reformats it from a 1D array of features into a 2D array with 1 row — the shape predict_proba() expects.

```
rf_model.predict_proba(sample)[0][1]
```

[0] gets the first (and only) row of results. [1] gets the second column — the probability of class 1 (Attack).

```
conf = prob_attack if pred == 'Attack' else 1 - prob_attack
```

Confidence is how sure the model is about whichever class it predicted. If predicting Attack: confidence = prob_attack. If predicting Normal: confidence = 1 - prob_attack (i.e. probability of Normal).

```
status = '■' if actual == pred else '■'
```

Simple check: did the model get it right? Prints a tick or cross for quick visual scan.

Mixed-Confidence Demo

An improvement over the random demo. Instead of picking any 5 rows, it deliberately picks rows where the model is only 55–90% confident — showing it actually weighing evidence rather than making trivially easy predictions.

```
all_probs = rf_model.predict_proba(X_test)[: , 1]

confidence_per_row = np.where(all_probs >= 0.5, all_probs, 1 - all_probs)

mixed_mask = (confidence_per_row >= 0.55) & (confidence_per_row <= 0.90)
mixed_idx = np.where(mixed_mask)[0]

np.random.seed(7)
chosen = np.random.choice(mixed_idx, 5, replace=False)
```

```
rf_model.predict_proba(X_test)[: , 1]
```

Gets the probability of Attack for EVERY test row at once (not row by row). [:, 1] selects the Attack column across all rows.

```
np.where(all_probs >= 0.5, all_probs, 1 - all_probs)
```

`np.where(condition, if_true, if_false)`: element-wise conditional. If the model predicts Attack (`prob >= 0.5`), `confidence = prob_attack`. Otherwise `confidence = 1 - prob_attack` (how sure it is it's Normal).

```
(confidence_per_row >= 0.55) & (confidence_per_row <= 0.90)
```

Boolean mask: True only where confidence is between 55% and 90%. The `&` operator combines two conditions — both must be True.

```
np.where(mixed_mask)[0]
```

Returns the row indices where the mask is True. [0] is needed because `np.where` returns a tuple — we want the first (and only) element.

```
np.random.choice(mixed_idx, 5, replace=False)
```

Picks 5 random indices from the filtered pool. If fewer than 5 rows qualify, the code falls back to using all available ones.

Why Test Accuracy is ~77% — Expected Behaviour

Students often worry that 77% test accuracy is low when training CV accuracy is ~99%. This section explains why the gap is completely expected and not a bug.

Short answer

KDDTest.csv intentionally contains attack types that never appear in KDDTrain.csv. This simulates real-world 'zero-day' attacks. No model can correctly classify a pattern it has never seen — and that's by design.

Dataset design intent	NSL-KDD was created specifically to test generalisation. The test set includes novel attack categories to simulate what real IDS systems face in production.
Evidence in the results	Attack recall is ~0.62 vs Normal recall ~0.97. The model is excellent at catching known attack patterns, but weaker on the unseen ones — exactly as expected.
Published benchmarks	Most published research on NSL-KDD binary classification (including deep learning) reports test accuracy in the 75–82% range. A test accuracy near 99% would actually be a red flag suggesting data leakage.
What was done about it	Threshold tuning (Section 5b) was applied to improve recall on the test set — trading a few extra false alarms for catching more real attacks.